МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу “Компьютерная графика”

Камера и освещение

Выполнила: Я. Р. Тарабрина
Группа: М8О-309БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: реализовать камеру с возможностью перемещения и освещение по модели Блинна-Фонга с несколькими типами источников света.

Вариант 10: правильный додекаэдр.

Программа написана на C++20 с использованием Vulkan, GLFW и Dear ImGui на основе стартового проекта и кода первой лабораторной работы. Додекаэдр освещается в реальном времени, а камерой, источниками света и материалами можно управлять из интерфейса.

Базовое задание: свободная камера (позиция, поворот, угол обзора), процедурные нормали, один направленный источник и рассеянный свет, материал (альбедо, цвет блика, коэффициент блика), модель Блинна-Фонга во фрагментном шейдере.

Выполнены все дополнительные задания:

1. внешние данные разделены на два набора дескрипторов: для сцены и для модели;
2. массив точечных источников в storage buffer, затухание по закону обратных квадратов, добавление и удаление источников в интерфейсе;
3. массив прожекторных источников в storage buffer, затухание по закону обратных квадратов, мягкие края освещённой области, добавление и удаление источников в интерфейсе;
4. показ источников света: яркие кубы и белый конус прожектора с осью, которые не освещаются по модели Блинна-Фонга и включаются в интерфейсе.

Кроме этого, на сцене есть пол и несколько объектов, которые движутся по траекториям из первой работы.

Исходный код: https://github.com/batsuperflash/cg-labs, папка `lab02`.

Метод решения

Камера

Камера свободно летает по сцене. Её состояние — позиция $\mathbf{e}$ и два угла: рыскание ψ (yaw, поворот вокруг оси Y) и тангаж φ (pitch, наклон вверх и вниз). Направление взгляда:

$$\mathbf{f} = (\sin\psi\cos\varphi,\ \sin\varphi,\ -\cos\psi\cos\varphi).$$

При $\psi = \varphi = 0$ камера смотрит вдоль оси $-Z$, положительный ψ поворачивает её направо, положительный φ — вверх. Тангаж ограничен диапазоном $[-89°, 89°]$, чтобы камера не переворачивалась и вектор вверх никогда не совпадал с направлением взгляда. Матрица вида строится написанной вручную функцией `lookAt` из первой работы: $V = \text{lookAt}(\mathbf{e}, \mathbf{e} + \mathbf{f}, \mathbf{up})$, где $\mathbf{up} = (0,1,0)$.

Управление: клавиши W и S двигают камеру вперёд и назад вдоль $\mathbf{f}$, A и D — вправо и влево, Q и E — вниз и вверх по оси Y, правая кнопка мыши с движением поворачивает камеру. Все параметры (позиция, ψ, φ, угол обзора, скорость, плоскости отсечения, тип проекции) дублируются в интерфейсе. Матрицы перспективной и ортографической проекции остались из первой работы.

Те же два угла задают направление направленного и прожекторного источников света, поэтому для них используется одна функция.

Нормали

Для освещения у каждой вершины есть нормаль. Чтобы грани додекаэдра освещались плоско, а рёбра были чёткими, нормаль должна быть одинаковой на всей грани. Поэтому у каждой из 12 граней свои 5 вершин: получается 60 вершин вместо 20, индексов по-прежнему 108.

Нормали генерируются процедурно. Вершины грани упорядочены против часовой стрелки, если смотреть снаружи, поэтому нормаль равна нормированному векторному произведению двух рёбер первого треугольника грани:

$$\mathbf{n} = \text{normalize}((\mathbf{b} - \mathbf{a}) \times (\mathbf{c} - \mathbf{a})).$$

Автоматический тест проверяет, что для каждого треугольника нормали всех трёх вершин равны этому произведению, что нормаль единичной длины и смотрит наружу и что различных нормалей ровно 12.

При переводе нормали в мировые координаты нельзя просто умножать её на матрицу модели: при неравномерном масштабе нормаль перестанет быть перпендикулярной поверхности. Нужна обратная транспонированная матрица $(M^{-1})^T$. Матрица модели имеет вид $M = T\,R\,S$, поэтому $(M^{-1})^T = R\,S^{-1}$: поворот сохраняется, а масштаб заменяется на обратный. Так матрица считается без обращения произвольных матриц, а тест сверяет её с `glm::inverseTranspose` и проверяет, что нормаль остаётся перпендикулярной растянутой поверхности. В шейдере нормаль после умножения нормируется заново, потому что масштаб меняет длину.

Модель Блинна-Фонга

Освещение считается во фрагментном шейдере для каждого пикселя в мировых координатах. Для точки с нормалью $\mathbf{N}$ вектор на камеру $\mathbf{V}$ и вектор на источник $\mathbf{L}$ задают «средний» вектор $\mathbf{H} = \mathrm{normalize}(\mathbf{L} + \mathbf{V})$. Цвет точки:

$$C = k_a\,A + \sum_i E_i\,(k_d \max(\langle \mathbf{N}, \mathbf{L}_i \rangle, 0) + k_s \max(\langle \mathbf{N}, \mathbf{H}_i \rangle, 0)^p),$$

где k_d — альбедо (цвет рассеянного отражения), k_s — цвет блика, p — коэффициент блика, A — рассеянный свет (цвет, умноженный на интенсивность), k_a — альбедо, E_i — освещённость от источника i (его цвет, умноженный на интенсивность, и затухание). Блик добавляется только если источник освещает поверхность, то есть $\langle \mathbf{N}, \mathbf{L} \rangle > 0$. Чем больше p, тем меньше и резче блик.

Все источники используют одну функцию `blinnPhong`, а различаются тем, как находят $\mathbf{L}$ и E:

- **направленный** источник (как Солнце): направление $\mathbf{d}$ одно для всей сцены, $\mathbf{L} = -\mathbf{d}$, затухания нет;
- **точечный** источник в точке $\mathbf{p}_l$: $\mathbf{L}$ направлен из точки сцены $\mathbf{p}$ на источник, $r = \| \mathbf{p}_l - \mathbf{p} \|$;
- **прожекторный** источник: то же, что точечный, но с множителем, зависящим от угла.

Затухание по закону обратных квадратов

Освещённость от точечного и прожекторного источника убывает пропорционально квадрату расстояния до него:

$$E = \frac{I \cdot \mathbf{c}}{r^2},$$

где I — интенсивность, $\mathbf{c}$ — цвет источника. Квадрат расстояния ограничен снизу малым числом, чтобы не делить на ноль, если точка совпала с источником.

Прожекторный источник и мягкие края

Прожектор задаётся позицией, направлением $\mathbf{d}$ (два угла, как у камеры), цветом, интенсивностью и двумя углами, отсчитываемыми от оси: внутренним θ_{in} и внешним θ_{out}. Для точки сцены берётся косинус угла между осью и лучом от прожектора к точке: $\cos\alpha = \langle -\mathbf{L}, \mathbf{d} \rangle$. Множитель края:

$$t = \mathrm{clamp}\left(\frac{\cos\alpha - \cos\theta_{out}}{\cos\theta_{in} - \cos\theta_{out}},\ 0,\ 1\right), \qquad s = t^2(3 - 2t).$$

Внутри внутреннего конуса $s = 1$, за внешним конусом $s = 0$, а между ними свет плавно затухает: сглаживающая функция $t^2(3 - 2t)$ не даёт заметного перехода на границах. Итоговая освещённость $E = s \cdot I\mathbf{c}/r^2$. Если углы равны, край резкий, а если внешний больше внутреннего, мягкий. В интерфейсе внутренний угол не может превысить внешний. Косинусы углов считаются на процессоре один раз, а шейдер сравнивает их с косинусом угла точки, поэтому тригонометрических функций в шейдере нет.

Данные для шейдеров: наборы дескрипторов и буферы

Внешние данные разделены на два набора дескрипторов по тому, как часто они меняются.

Набор 0 — сцена, привязывается один раз за кадр:

- слот 0, uniform buffer `SceneUniforms` (208 байт): матрицы вида и проекции, позиция камеры, рассеянный свет, направление и цвет направленного источника, число точечных и прожекторных источников;

- слот 1, storage buffer с массивом точечных источников, элемент `{ position, radiance }` (32 байта);
- слот 2, storage buffer с массивом прожекторных источников, элемент `{ position, direction, radiance, cone }` (64 байта).

Набор 1 — модель, привязывается перед каждым вызовом рисования:

- слот 0, uniform buffer `ModelUniforms` (160 байт): матрица модели, матрица нормалей, альбедо, цвет блика и коэффициент блика.

В шейдерах набор сцены имеет номер `set = 0`, набор модели — `set = 1`. Соответствие задаёт порядок раскладок наборов в `VkPipelineLayout`. Команда `vkCmdBindDescriptorSets` с параметром `firstSet = 1` подставляет набор модели, не трогая уже привязанный набор сцены.

Массивы источников лежат в storage buffer, поэтому в шейдере у них нет размера: `PointLight lights[]`. Сколько элементов использовать, шейдер узнаёт из счётчиков в `SceneUniforms`. На стороне процессора буфер создан с запасом на 16 источников каждого вида, а в каждом кадре копируется только то количество элементов, которое есть в сцене. Благодаря этому максимум зашит только в размер буфера, но не в код шейдера.

Выравнивание. Uniform buffer использует раскладку std140: матрицы и `vec4` занимают по 16 байт, а `vec3` тоже занимает 16 байт, хотя в нём 12. Поэтому в структурах нет `vec3`: позиция камеры, цвета и направления хранятся как `vec4`, а коэффициент блика лежит в четвёртой компоненте цвета блика. Матрица нормалей хранится как `mat4`, так как `mat3` в std140 состоит из трёх столбцов по 16 байт и легко ошибиться в смещениях. Для storage buffer используется раскладка std430, элементы массивов состоят только из `vec4`, поэтому в них нет пропусков. Размеры и смещения проверяются через `static_assert`: 208 и 160 байт для uniform buffer, смещения 128 и 192 в `SceneUniforms`, 32 и 64 байта для элементов массивов.

Слоты моделей. Для каждого рисования модели берётся свой слот: свой uniform buffer и свой набор дескрипторов. Команды выполняются на GPU позже, чем записываются, поэтому один общий буфер хранил бы данные последней модели. Слотов создаётся 57: 8 объектов, пол, 16 значков точечных источников, 16 значков прожекторов и 16 конусов. Все

буферы и наборы создаются при запуске, во время работы ничего не создаётся и не удаляется. Как и в первой работе, данные записываются в функции `render()`, после ожидания предыдущего кадра.

Визуализация источников света

Источники света можно сделать видимыми. Для этого строятся два вспомогательных меша:

- куб с нормалями граней, 24 вершины; он ставится в точку каждого точечного и прожекторного источника и закрашивается цветом источника;
- конус из линий: вершина в начале координат, окружность основания радиуса 1 в плоскости $z = -1$, четыре образующие и ось. Матрица $T(\mathbf{p})\, R_y(-\psi)\, R_x(\varphi)\, S(r, r, L)$ ставит вершину в позицию прожектора, поворачивает ось $-Z$ в направление света и растягивает конус: длина $L = 3$, радиус основания $r = L\tan\theta_{out}$ соответствует внешнему углу.

Эти фигуры должны иметь постоянный цвет и не освещаться. Для этого есть второй фрагментный шейдер, который выводит цвет модели как есть, и два дополнительных графических конвейера с тем же вершинным шейдером: один рисует треугольники (кубы), другой — линии `LINE_LIST` (конусы). Кадр рисуется в таком порядке: освещаемые модели (объекты и пол), значки источников, конусы. Показ значков и конусов включается отдельными флажками в интерфейсе.

Используемые объекты Vulkan

- **VkBuffer и VmaAllocation**: вершинные и индексные буферы четырёх мешей (додекаэдр, пол, куб, конус), uniform buffer сцены, 57 uniform buffers моделей и два storage buffers источников.
- **VkDescriptorSetLayout**: две раскладки, сцены (uniform buffer и два storage buffers для фрагментного шейдера) и модели (uniform buffer).
- **VkPipelineLayout**: раскладки обоих наборов.
- **VkDescriptorPool и VkDescriptorSet**: один набор сцены и 57 наборов моделей, пул на 58 наборов (58 uniform buffers и 2 storage buffers).

- **VkShaderModule**: вершинный шейдер `mesh.vert`, фрагментные шейдеры `mesh.frag` (модель Блинна-Фонга) и `flat.frag` (постоянный цвет).
- **VkPipeline**: три конвейера. Все с тестом глубины, отсечением задних граней и динамическими viewport и scissor. Атрибуты вершины: позиция и нормаль (по 12 байт, шаг 24 байта).
- **VkRenderPass и VkFramebuffer** из стартового проекта, **VkCommandBuffer**: в него каждый кадр записываются привязка набора сцены, смена конвейеров и для каждой модели привязка буферов и набора модели с вызовом `vkCmdDrawIndexed`.

Результаты

На рисунке 1 показана базовая часть: три додекаэдра и пол освещены одним направленным источником и рассеянным светом. Грани освещены плоско, у каждой своя яркость в зависимости от угла между нормалью и направлением света, у красного объекта слева на грани, обращённой к источнику, виден блик.

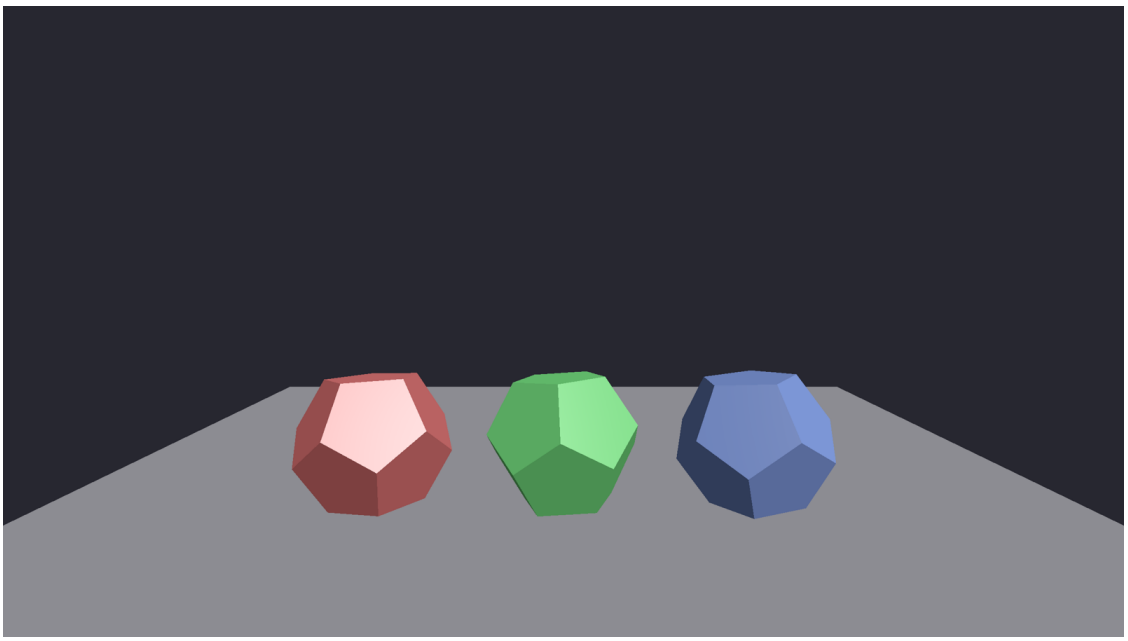


Рисунок 1 — Направленный источник и рассеянный свет

На рисунке 2 у трёх одинаковых красных объектов разный коэффициент блика: 4, 32 и 256. Блик виден только на тех гранях, нормаль которых близка к вектору **H**, а из-за плоских граней он занимает грань целиком. При малом коэффициенте (слева) блик широкий: освещённая грань почти белая и заметно светлее остальных. При среднем (в центре)

блик слабее, а при большом (справа) он стал настолько узким, что ни одна грань не попала в него, и видно только рассеянное отражение.

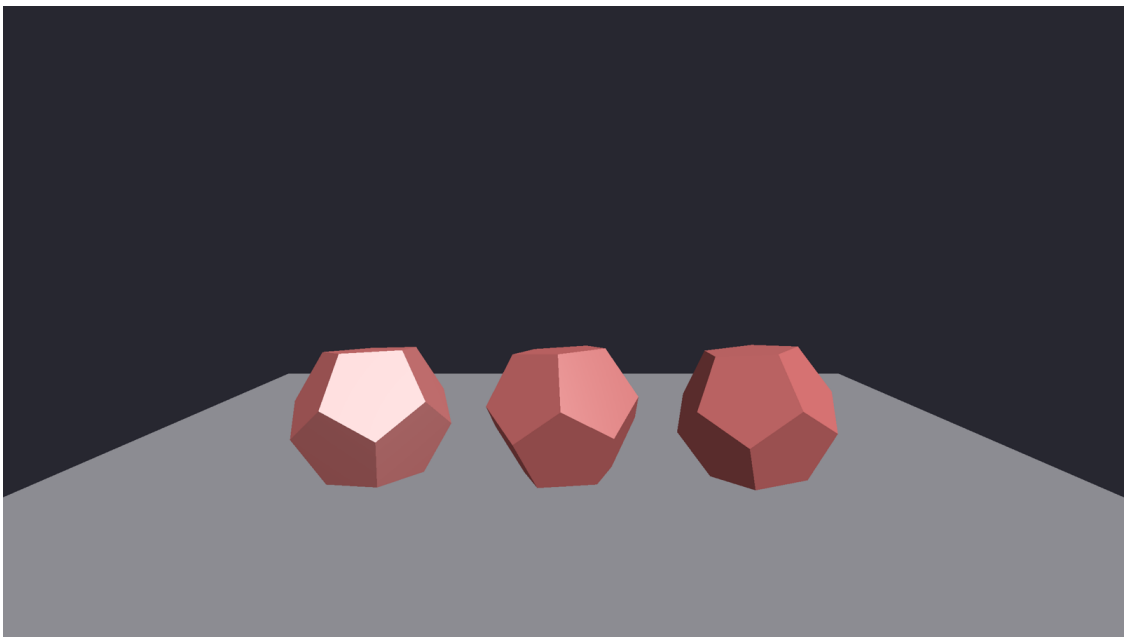


Рисунок 2 — Коэффициент блика 4, 32 и 256

На рисунке 3 направленный источник выключен, на сцене два точечных источника: красный слева и синий справа. Кубы показывают их положение. Свет убывает с расстоянием, поэтому пол ярче у источников, а в середине цвета смешиваются.

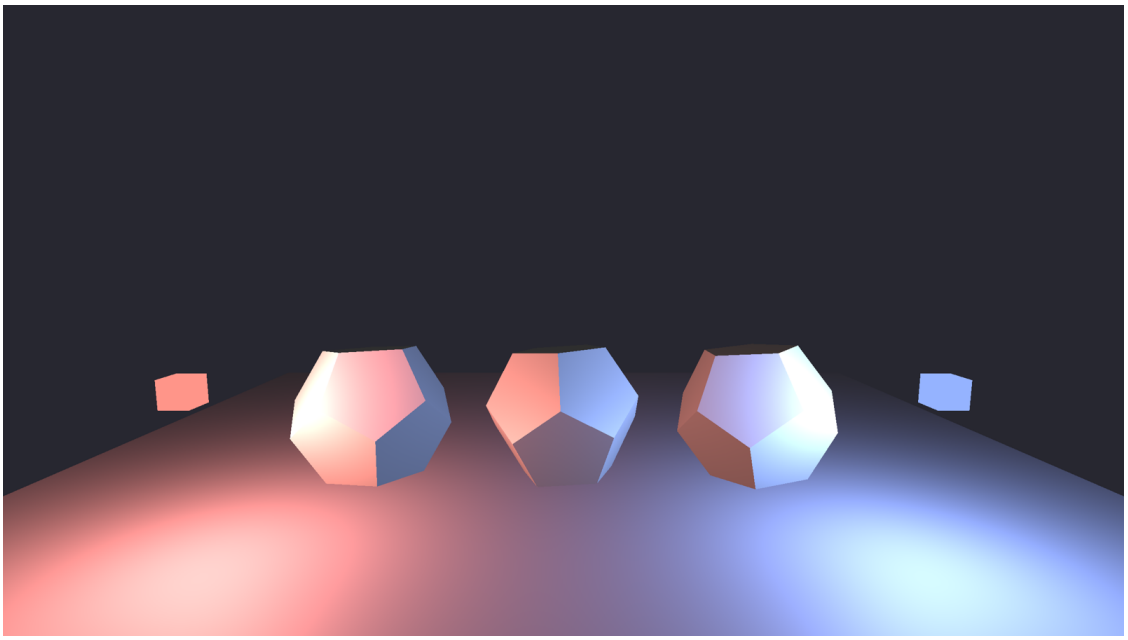


Рисунок 3 — Два точечных источника

На рисунке 4 два прожектора направлены вниз. У левого внутренний угол 6°, внешний 30°: край пятна на полу плавно затухает. У правого оба

угла равны 20°: край резкий. Белые линии показывают конус и ось, а яркие кубы цвета источников стоят в вершинах конусов. Части додекаэдров вне конусов остаются тёмными: направленный источник выключен, остаётся только слабый рассеянный свет.

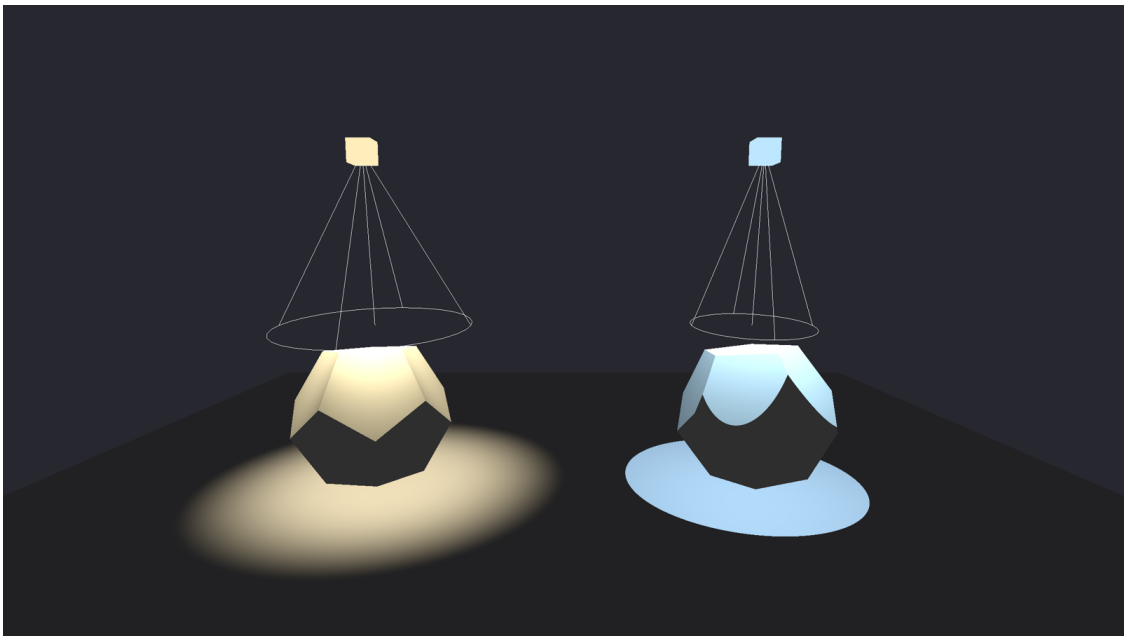


Рисунок 4 — Прожекторы с мягким и резким краем

На рисунках 5 и 6 показан интерфейс при начальной сцене: два точечных источника, прожектор, направленный и рассеянный свет. На рисунке 5 открыты разделы камеры и света, на рисунке 6 — разделы точечных и прожекторных источников, где можно добавлять и удалять источники и менять их параметры, и показ значков.

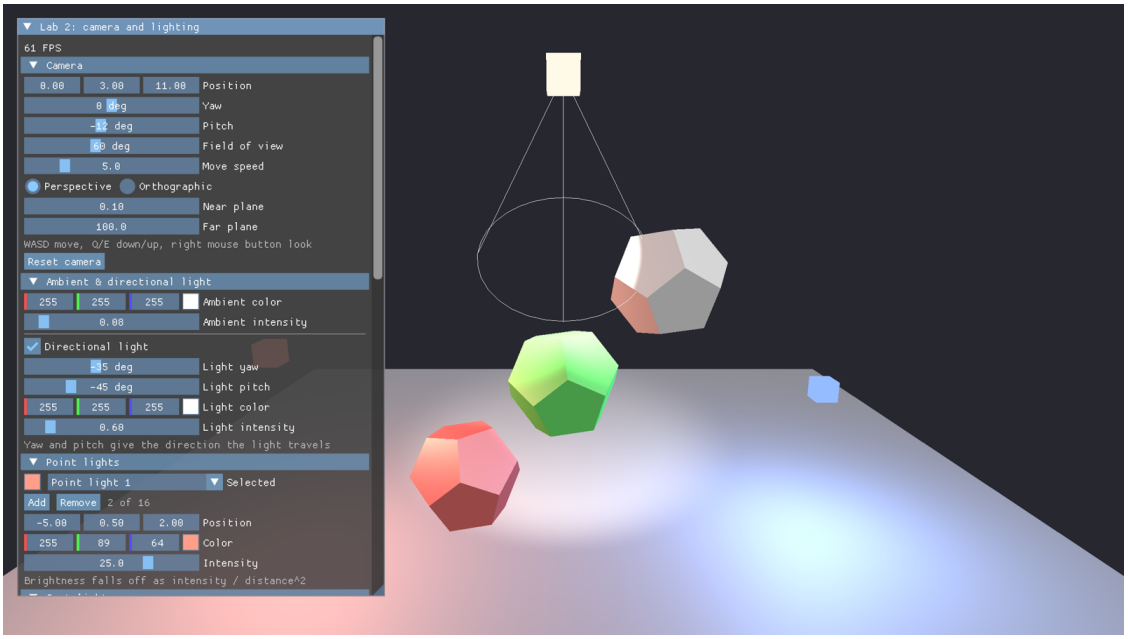


Рисунок 5 — Разделы Camera и Ambient & directional light

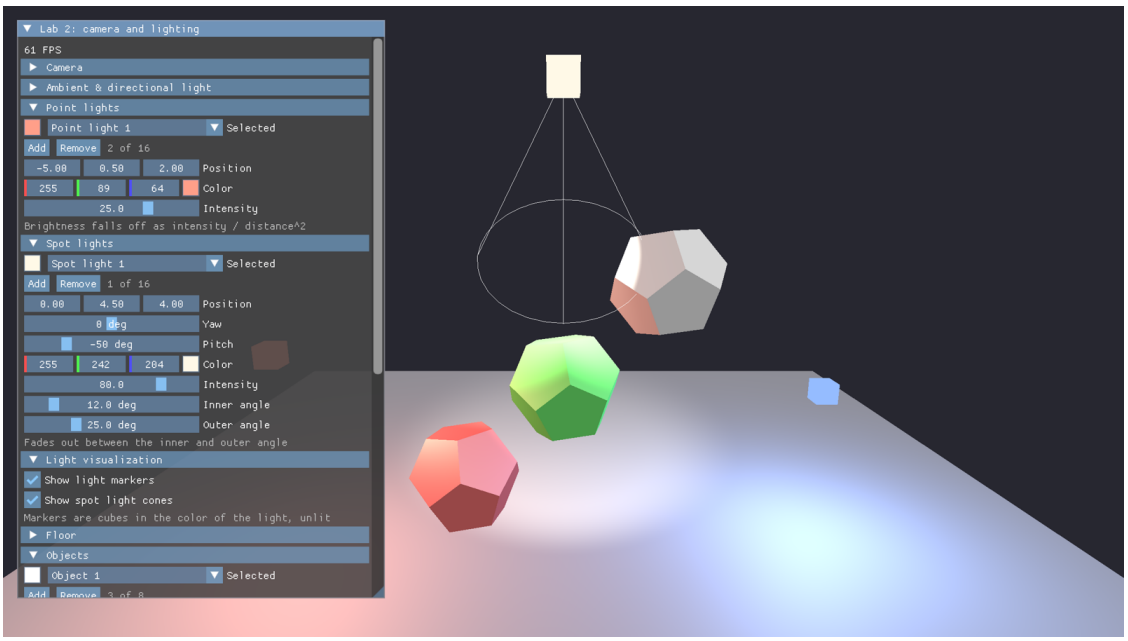


Рисунок 6 — Разделы Point lights, Spot lights и Light visualization

Автоматические тесты (496 проверок) проходят, а слой валидации Vulkan не выдаёт сообщений ни во время работы, ни при завершении программы. Тесты проверяют геометрию додекаэдра, нормали, матрицу нормалей, камеру (направление по углам, матрицу вида, движение, ограничение тангажа), добавление и удаление источников и матрицы значков источников.

Выводы

Я научилась строить вершинные нормали процедурно и объяснила себе, почему для плоского освещения граней вершины нельзя делить между гранями, а для нормалей при растяжении нужна матрица $R\,S^{-1}$, а не матрица модели. Я реализовала модель Блинна-Фонга для направленных, точечных и прожекторных источников и разобралась, как закон обратных квадратов и сглаживание по углу превращаются в несколько строк шейдера.

На стороне Vulkan я разделила данные на два набора дескрипторов по частоте изменения, передала массивы источников через storage buffers с размером, который не задан в шейдере, и разобралась с различиями выравнивания std140 и std430. Добавление собственных конвейеров для значков показало, что вершинный шейдер можно использовать повторно, а различия в шейдере и топологии делают рисование освещаемых и неосвещаемых объектов независимым.